

🔗🔗 Table of Contents

- 1. What is MongoDB? 2. Core Concepts
- 3. Database Commands 4. INSERT (Create)
- 5. READ (Find) 6. Comparison Operators
- 7. LOGICAL Operators 8. UPDATE
- 9. DELETE 10. Mini Project

What is MongoDB?

MongoDB is a **NoSQL database** — meaning it does NOT use tables and rows like traditional SQL databases. Instead, it stores data as **documents** (like JSON objects). It is flexible, fast, and perfect for modern web development (used in the MERN stack).

🔗🔗 Think of MongoDB like a folder of notebooks. Each notebook is a **Collection**, and each page in it is a **Document**.

Database Database schoolDB Table

Collection students Row / Record Document { name: "Ali", age: 20 } Column Field name, age, city

Quick Check — Section 1

What is the difference between SQL and MongoDB?

What is a Document in MongoDB? Give your own example.

Name the MongoDB equivalent of: Table, Row, Column.

Core Terms with Examples

Database

A container that holds all your collections. Like a cupboard that stores all your files.

EXAMPLE

schoolDB ← this is a database name

🔗🔗 Document

A single record (like one student's data). Stored in JSON-like format.

EXAMPLE

```
name: "Ali"
age: 20
city: "Karachi"
}
```

🔗🔗 collections Show all collections in current database

🔗🔗 db.createCollection("students") Create a new collection called students

FULL EXAMPLE — SETTING UP SCHOOLDB

// Step 1: See existing databases

show dbs

// Step 2: Create/switch to our database

use schoolDB

// Step 3: Confirm we are inside it

db // Output: schoolDB

// Step 4: Create a collection

db.createCollection("students")

// Step 5: Confirm collection was created

show collections

🔗🔗 Collection

A group of similar documents. Like a folder that holds student records.

EXAMPLE

students ← this is a collection

Database & Collection Commands

🔗🔗 Field

A key-value pair inside a document. Like a label and its value. EXAMPLE

name: "Ali" ← field

age: 20 ← field

city: "Karachi" ← field

Basic Commands

🔗🔗 show dbs Show all databases in MongoDB

🔗🔗 use schoolDB Create or switch to a database named schoolDB

🔗🔗 db Show the currently active database

INSERT Operations (Create)

🔗🔗 In CRUD, **C = Create**. We use insertOne() to add one document, and insertMany() to add multiple at once.

insertOne() — Add a single document

Use when you want to insert exactly one student/product/record.

```
// Syntax
db.collectionName.insertOne({ field: value })

// Example — Insert one student
db.students.insertOne(
  { name: "Ali",
    age: 20,
    city: "Karachi",
    course: "Web Dev"
  })

// MongoDB auto-creates a unique _id for each document
// Output: { acknowledged: true, insertedId: ObjectId("..." ) }
```

insertMany() — Add multiple documents

Use when you want to insert a batch of records at once. Pass an array [] of documents.

```
// Syntax
db.collectionName.insertMany([ {}, {}, {} ])

// Example — Insert 4 students at once
db.students.insertMany(
  [ { name: "Ahmed", age: 22, city: "Hyderabad", course: "Python" },
    { name: "Sara", age: 19, city: "Sukkur", course: "UI/UX" },
    { name: "Bilal", age: 24, city: "Karachi", course: "Node.js" },
    { name: "Hina", age: 21, city: "Lahore", course: "React" }
  ])

⚠️ Notice the square brackets [] — insertMany requires an array! Forgetting brackets is a common mistake.
```

Assignment 2 — INSERT Operations

Use insertOne() to add yourself as a student (name, age, city, course).

Use insertMany() to add 5 more classmates with different cities.

Add a student with an extra field called fee: 5000 .

Insert 3 products into a products collection with name, price, category.

READ Operations (Find Data)

🔗🔗 In CRUD, **R = Read**. We use find() and findOne() to search and retrieve documents from a collection.

find() — Get all or filtered documents

```
// 1. Show ALL documents (no filter)
db.students.find()

// 2. Pretty format (easier to read)
db.students.find().pretty()

// 3. Find FIRST document only
db.students.findOne()

// 4. Find by specific field value
db.students.find({ name: "Ali" })

// 5. Find students from Karachi
db.students.find({ city: "Karachi" })

// 6. Find by multiple fields (AND condition)
db.students.find({ city: "Karachi", age: 20 })

// 7. Count total documents
db.students.countDocuments()
```

🔗🔗 When you pass nothing to find(), it returns ALL documents. When you pass { field: value }, it filters results.

Assignment 3 — READ Operations

Show all students in the collection.

Find only the first student using findOne() .

Find all students from your city.

Find a student by their name.

Find students from Karachi who are doing "Web Dev" course (two conditions).

Count total number of students in the collection.

Comparison Operators

🔗🔗 These operators let you filter data based on numeric comparisons. They always go inside curly braces as { field: { Operator: value } }.

All Comparison Operators with Examples

🔗🔗 Greater Than — students older than 20

```
db.students.find({ age: { $gt: 20 } })
// Returns: Ahmed(22), Bilal(24), Hina(21)
```

🔗🔗 Less Than — students younger than 22

```
db.students.find({ age: { $lt: 22 } })
// Returns: Ali(20), Sara(19), Hina(21)
```

🔗🔗 Greater Than or Equal — age 20 and above

```
db.students.find({ age: { $gte: 20 } })
// Returns: Ali(20), Ahmed(22), Bilal(24), Hina(21)
```

🔗🔗 Less Than or Equal — age 21 and below

```
db.students.find({ age: { $lte: 21 } })
// Returns: Ali(20), Sara(19), Hina(21)
```

🔗🔗 Not Equal — students NOT from Karachi

```
db.students.find({ city: { $ne: "Karachi" } })
// Returns: Ahmed(Hyderabad), Sara(Sukkur), Hina(Lahore)
```

🔗🔗 All operators start with \$ — this is how MongoDB knows it's an operator, not a regular field name.

Assignment 4 — Comparison Operators

Find all students older than 21 using \$gt .

Find students younger than 20 using \$lt .

Find students aged 19 or older using \$gte .

Find students aged 22 or younger using \$lte .

Find all students NOT doing "Python" course using \$ne .

Find students with fee greater than or equal to 4000 using \$gte .

Logical Operators

🔗🔗 Logical operators combine multiple conditions. \$or returns results matching ANY condition. \$in checks if a value exists in a list.

\$or — Match any one of the conditions

Use when you want documents that match condition A OR condition B.

```
// Find students from Karachi OR Sukkur
db.students.find(
  { $or: [
    { city: "Karachi" },
    { city: "Sukkur" }
  ] })
// Returns: Ali(Karachi), Bilal(Karachi), Sara(Sukkur)
```

\$in — Match if value is in a list

Use when you have a list of values and want any document matching one of them. Shorter than using multiple \$or conditions.

```
// Find students from Karachi, Hyderabad, OR Sukkur
db.students.find(
  { city: { $in: ["Karachi", "Hyderabad", "Sukkur"] } })
// Returns: Ali, Ahmed, Sara, Bilal
```

\$nin — NOT in a list

The opposite of \$in. Returns documents where the field value is NOT in your list.

```
// Find students NOT from Karachi or Hyderabad
db.students.find(
  { city: { $nin: ["Karachi", "Hyderabad"] } })
// Returns: Sara(Sukkur), Hina(Lahore)
```

Assignment 5 — Logical Operators

Use \$or to find students from Lahore or Sukkur.

Use \$in to find students doing "Web Dev", "React", or "Node.js".

Use \$nin to find students NOT doing "Python" or "UI/UX".

Use \$or with age: find students younger than 20 OR older than 22.

UPDATE Operations

🔗🔗 In CRUD, **U = Update**. Always use \$set to change values. Without it, MongoDB will replace the ENTIRE document!

updateOne() — Update a single document

```
// Change Ali's city from Karachi to Lahore
db.students.updateOne(
  { name: "Ali" }, ← filter: find this document
  { $set: { city: "Lahore" } } ← what to change
)
// Output: { matchedCount: 1, modifiedCount: 1 }
```

updateMany() — Update multiple documents

```
// Change ALL Karachi students to Hyderabad
db.students.updateMany(
  { city: "Karachi" },
  { $set: { city: "Hyderabad" } })

// Add new field "course" to Ali's document
db.students.updateOne(
  { name: "Ali" },
  { $set: { course: "Web Development" } })

// Rename field "city" to "location" in ALL documents
db.students.updateMany(
  {}, ← {} means all documents
  { $rename: { city: "location" } })

// Remove "course" field from Ali's document
db.students.updateOne(
  { name: "Ali" },
  { $unset: { course: "" } })
```

⚠️ \$set = Change/Add a field | \$rename = Rename a field | \$unset = Delete a field from document

Assignment 6 — UPDATE Operations

Update your own city to a new city using updateOne() .

Change the course of ALL students from "Python" to "AI" using updateMany() .

Add a new field grade: "A" to a specific student.

Rename the grade course to subject in all documents.

Remove the grade field from one student using \$unset .

DELETE Operations

⚠️ In CRUD, **D = Delete**. Be very careful — deleted data cannot be recovered easily! Always double-check your filter before deleting.

All Delete Operations with Examples

```
// 1. Delete ONE document (only first match)
db.students.deleteOne({ name: "Ali" })
// Output: { deletedCount: 1 }

// 2. Delete MANY documents matching a filter
db.students.deleteMany({ city: "Karachi" })
// Deletes ALL students from Karachi

// 3. Delete ALL documents (collection stays empty)
db.students.deleteMany({}) ← {} = no filter = all

// 4. Drop entire collection (collection is deleted)
db.students.drop()

// 5. Drop entire database (everything deleted)
db.dropDatabase()
```

deleteOne(filter) First matching document ✅ Yes

deleteMany(filter) All matching documents ✅ Yes

deleteMany({}) All documents ✅ Yes (but empty) drop() Entire collection ❌ No

dropDatabase() Entire database ❌ No

Assignment 7 — DELETE Operations

Delete one student by name using deleteOne() .

Delete all students from a specific city using deleteMany() .

Insert 5 new students, then delete all documents using deleteMany({}) .

Drop the students collection using drop() .

What is the difference between deleteMany({}) and drop() ? Write in your own words.

🔗🔗 Final Mini Project — Student Management System

Apply everything you've learned. Build a complete MongoDB project from scratch.

REQUIRED FIELDS

name age city

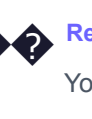
course fee grade

- ◆ Insert at least 10 students using insertMany()
- ◆ Find all students from your city
- ◆ Find students with fee greater than 3000 using \$gt
- ◆ Find students doing "Web Dev" OR "React" using \$or or \$in
- ◆ Update one student's course and city
- ◆ Add a new field "batch" to all students using updateMany()
- ◆ Delete students from a specific city
- ◆ Show filtered results using \$lt, \$gte, \$nin operators



MongoDB Basics

You now understand what MongoDB is, how it compares to SQL, and all core terms.



Query Operators



CRUD Operations



You can Create, Read, Update, and Delete documents in any collection.

You can filter data using \$gt, \$lt, \$in, \$or, \$ne and more.



Real-World Skills

You can handle real databases like a backend developer in the MERN stack.

TITANS SUKKUR

Sir Yasir Ali Lashari — MongoDB Complete Notes